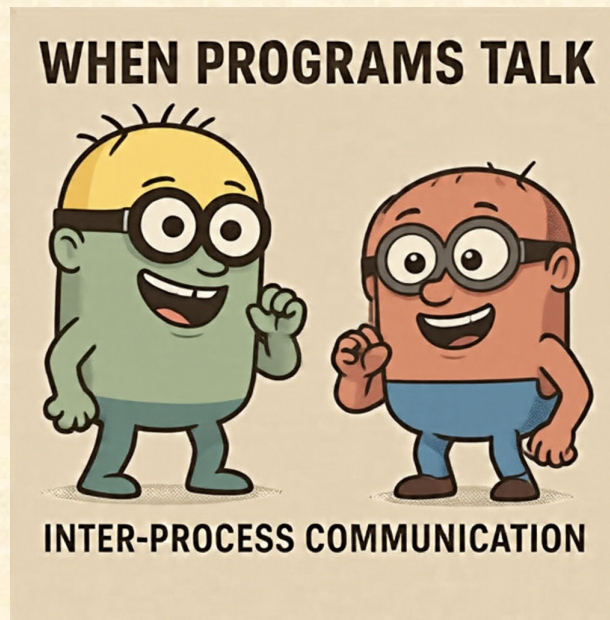


Inter-Process Communication

Galvin Ch3.4



Sk Subidh Ali
IIT Bhilai

Inter-process communication

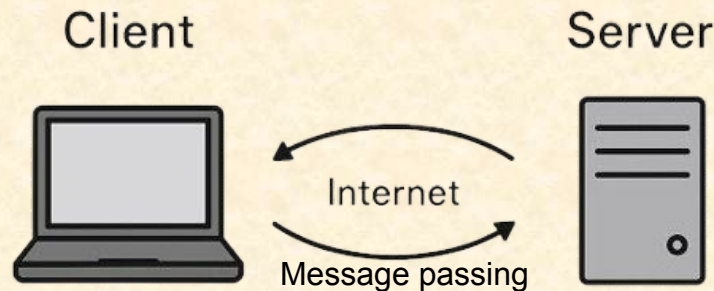
IPC: OS's information sharing mechanism for processes

While developing software a job is divided into multiple tasks

- A task can be accomplished using a single process
- Multiple processes cooperate to perform the job

Client server:

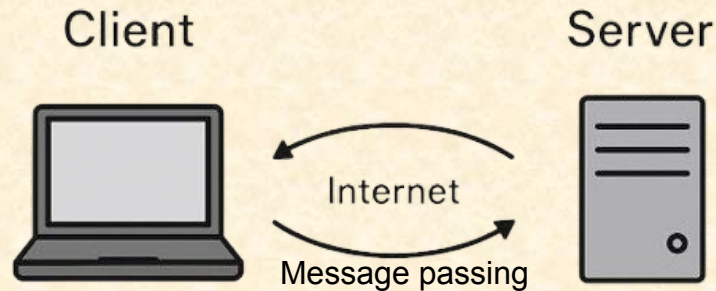
- Client: Request to server through message
- Server: Sends the response



Application: Client-Server

Client server:

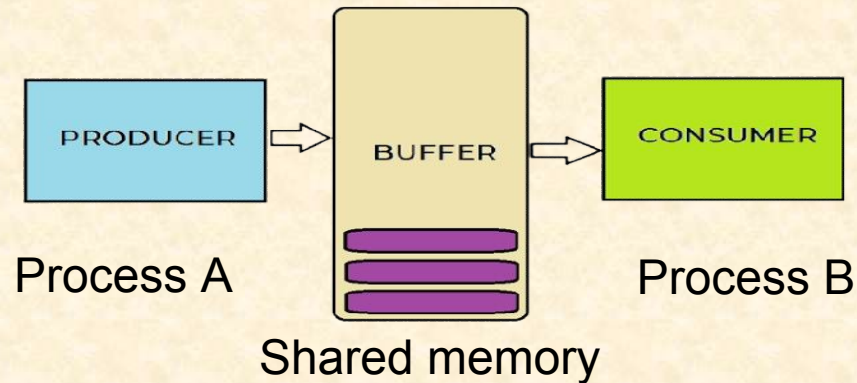
- Client: Request to server through message
- Server: Sends the response



Application: Producer-Consumer

Producer consumer job:

- Producer: Produce something and store it for consumer
- Consumer: Consume the produced item

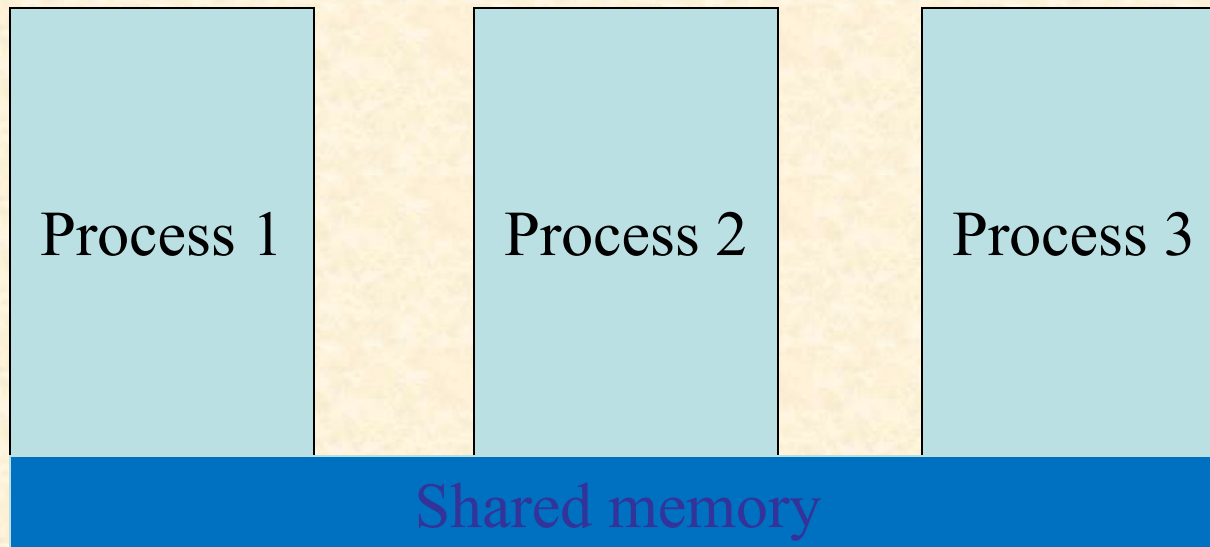


Basic mechanism for IPS

- Shared memory:
- Message Passing:

IPS with shared memory

- **Shared memory:** OS provides a memory sharing mechanism
 - Processes exchange data by reading or writing to shared memory



Shared Memory in VM

P

0	0
	1
	2
	3

Virtual address space of P_0

0	2
1	7
2	5
3	10

PT_0

P

1	0
	1
	2
	3
	4

Virtual address space of P_0

0	0
1	1
2	5
3	3
4	11

PT_1

RAM

0	
1	
2	
3	
4	
5	SharedMemory
6	
7	
8	
9	
10	
11	

Frames in RAM

The shared memory frame mapped to page tables of process involved in the IPC

Shared Memory in Solaris

- System calls:

int shmget(key_t key, size_t size, int shmflg) :

Returns a memory segment id

Size of the memory segment

Permission bit (0666)

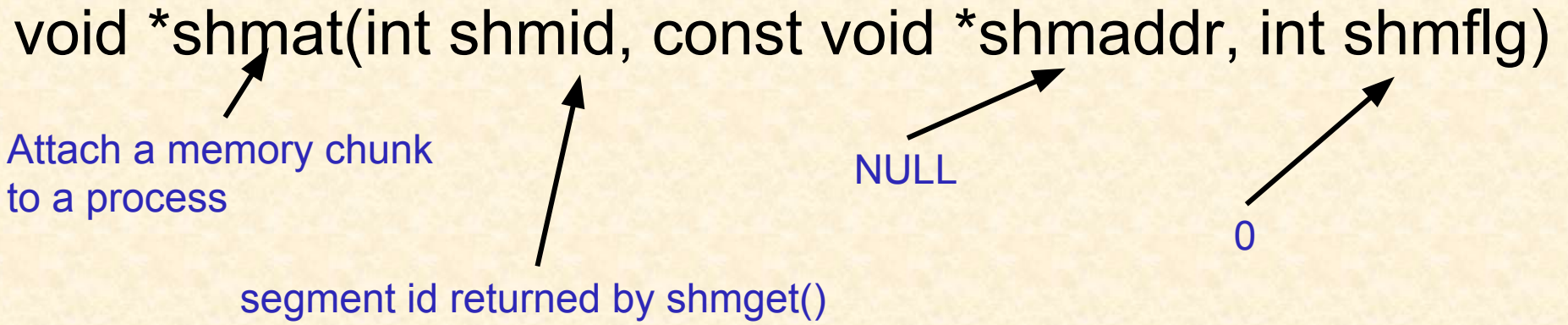
The memory segment uniquely identified by **key**

Shared Memory in Solaris

- System calls:

`void *shmat(int shmid, const void *shmaddr, int shmflg)`

Attach a memory chunk
to a process



segment id returned by shmget()

NULL

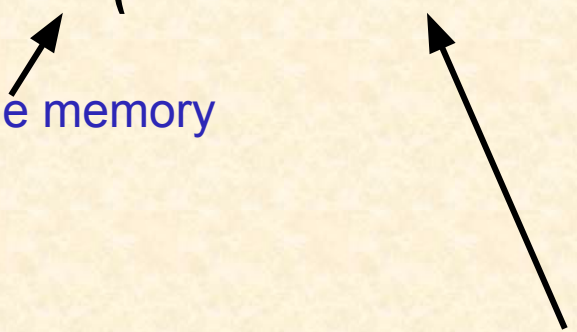
0

Shared Memory in Solaris

- System calls:

*int shmdt(char *shmaddr)*

Detach the memory
segment



The diagram consists of two black arrows. One arrow originates from the text 'Detach the memory segment' and points to the 'shmaddr' parameter in the function signature. The other arrow originates from the text 'segment id' and points to the 'char *' part of the 'shmaddr' parameter.

segment id

Shared memory example

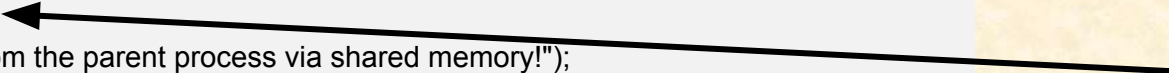
```
int main(void) {
    int shmid = shmget(IPC_PRIVATE, 100 , IPC_CREAT | 0666);
    char *ch = (char *) shmat(shmid, NULL, 0);
    ch[] = '\0';
    pid_t pid = fork();

    if (pid == 0) {
        printf("[child] waiting for data...\n");
        printf("[child] read from shared memory: \"%s\"\n", ch);
        shmdt(shm);
        exit(0);
    } else {
        strcpy(ch, "Hello from the parent process via shared memory!");
        printf("[parent] wrote to shared memory, signaling child...\n");
        wait(NULL); // wait for child to finish
        shmdt(shm);
        shmctl(shmid, IPC_RMID, NULL); // mark segment for destruction
        printf("[parent] segment removed. Done.\n");
    }
    return 0;
}
```

Child read from shared memory



Parent writes to shared memory



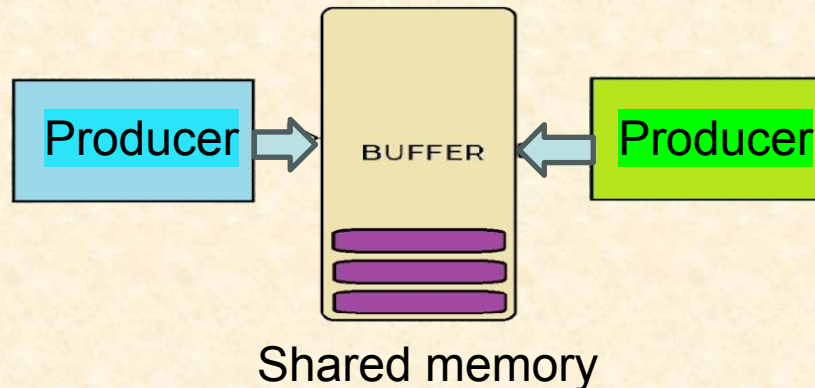
```
$/a.out
[parent] wrote.....
[child] waiting for ...
[child] read from..
...: Hello from ...
```

IPC_PRIVATE: shared memory is exclusive for current process and its fork() child

Challenges in Shared Memory

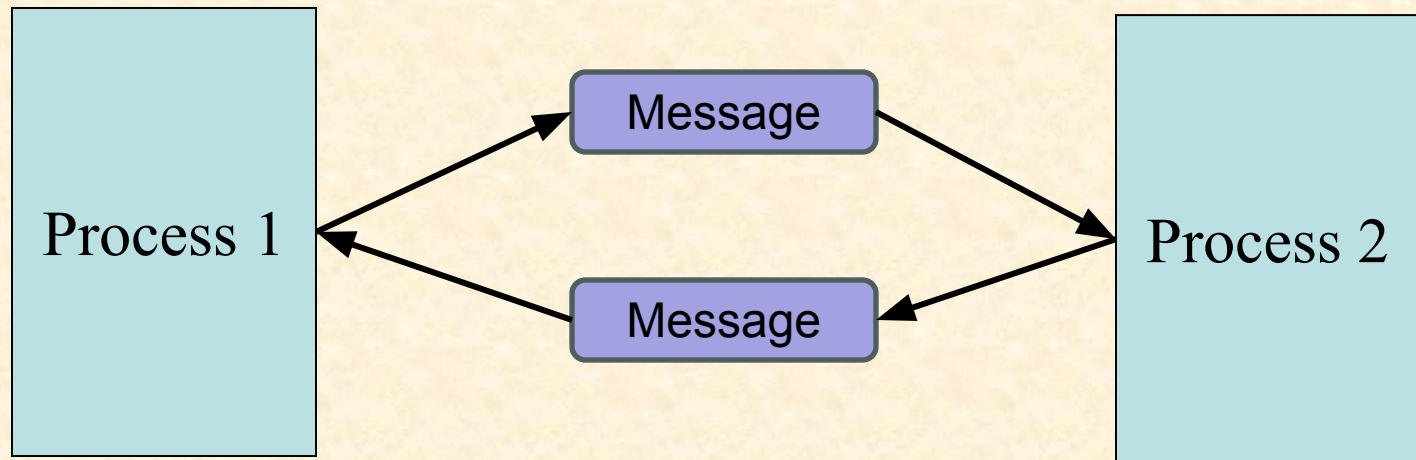
If more than one process writes:

- May lead to race condition
- Writing processes must be moderated
 - **Process synchronization:**
 - One writes other waits



IPS with message passing

- Message passing :
 - Processes exchange data via explicit OS system calls `read()` or `write()`



IPS with message passing

- Message passing :
 - Processes exchange data via explicit OS system calls `read()` or `write()`

Pipe	Unidirectional byte stream between related processes. Example: <code>echo test.c wc -w</code>
Socket	Endpoint for communication, works both locally and across a network (TCP/IP sockets)
Signal	Asynchronous notification of an event (not really data transfer) — e.g., <code>SIGKILL</code> , <code>SIGCHLD</code> ; interrupts the receiving process to run a handler or take a default action

Pipes

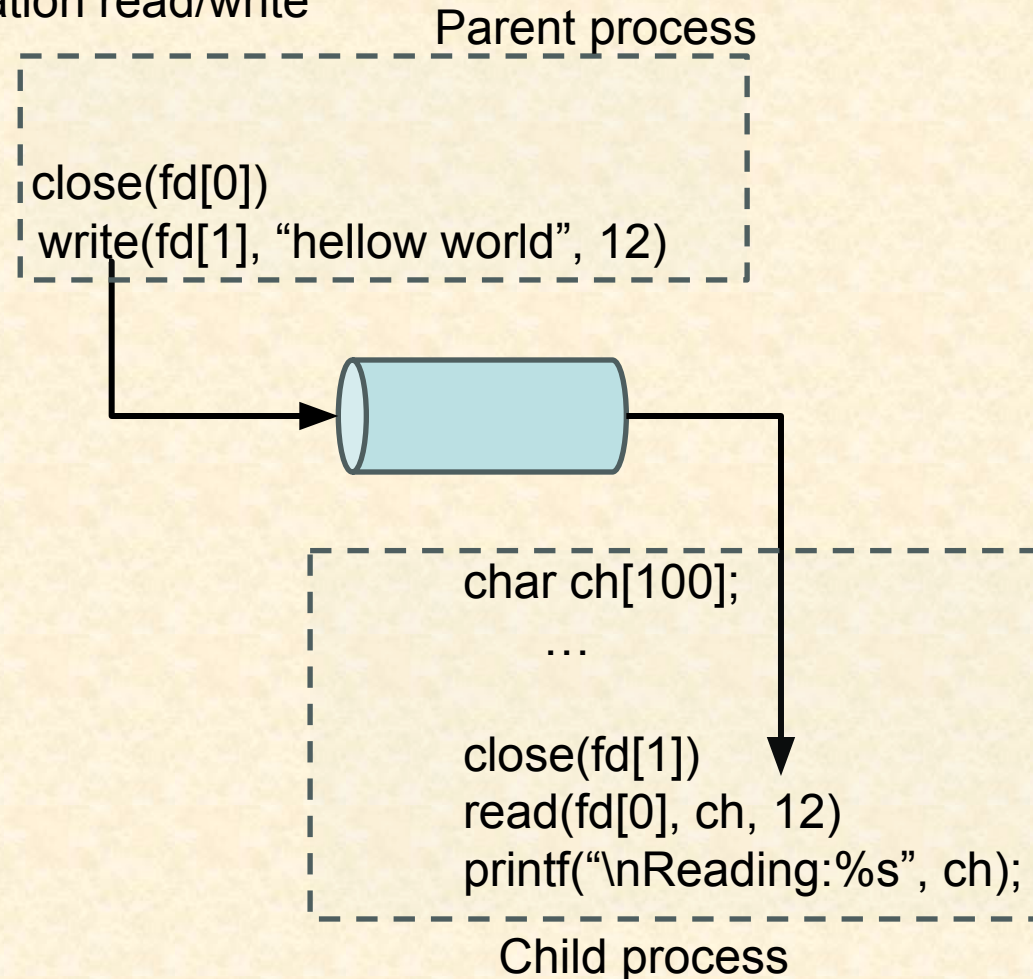
A **pipe** is the simplest form of IPC in Linux

- Provide unidirectional communication read/write

```
int pipe(int fd[2]);
```

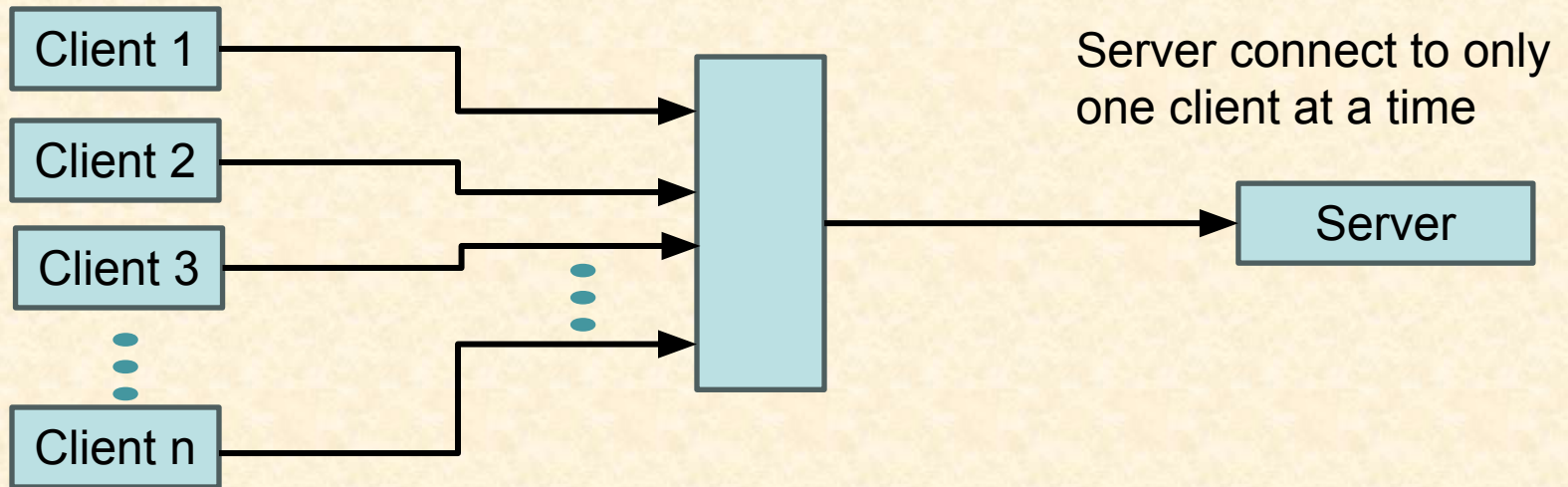
Provides two file descriptors:

- fd[0] for reading from the pipe
- fd[1] for writing to the pipe



Socket

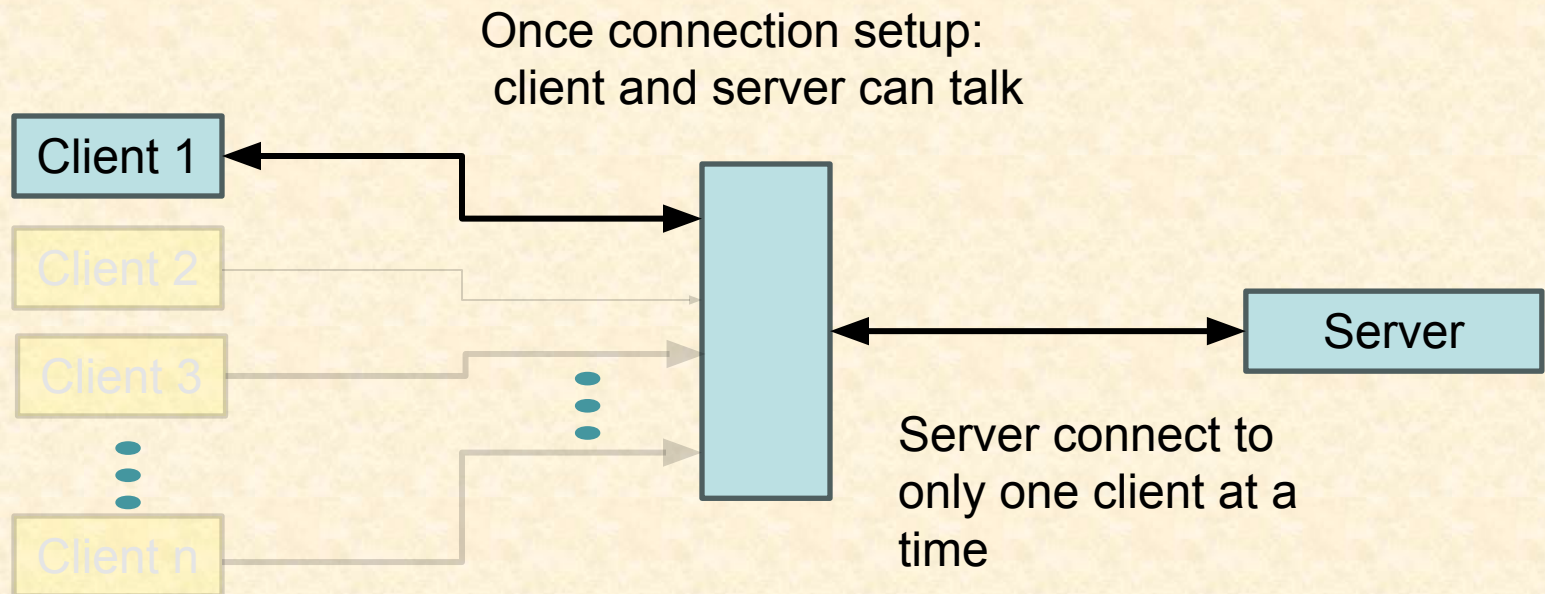
- **Socket** is widely used IPC
- Almost all client-servers are built using socket
- **communication** can happen
 - within a machine
 - across network



Every client try to connect to server

Socket

- **Socket** is widely used IPC
- Almost all client-servers are built using socket
- **communication** can happen
 - within a machine
 - across network



Socket

Socket()	<i>Returns a file descriptor to the callee process</i>
bind()	Attach it to a specific local address (IP+port, or filesystem path for Unix sockets)
listen()	Server listen to for accept incoming connections, with a backlog queue
accept()	Block until a client connects; returns a new socket fd dedicated to that one connection (the original listening socket keeps listening for more)

Lets see the code and build a client and server using socket

Socket

```
int main(int argc, char const *argv[]) {
    int fd = socket(AF_UNIX, SOCK_STREAM, 0); // 1. create

    struct sockaddr_un addr;

    addr.sun_family = AF_INET;
    addr.sun_port=htons(8000) //Server Port no is 8000

    if (inet_pton(AF_INET, argv[1], &addr.sin_addr) <= 0) {
        printf("\nInvalid address/ Address not supported\n");
        return -1; }

    if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        perror("connect"); exit(1);
    }

    const char *msg = "Hello over a socket!";
    write(fd, msg, strlen(msg)); // 3. send

    char buf[300];
    int n = read(fd, buf, sizeof(buf) - 1); // 4. receive reply
    buf[n] = '\0';
    printf("[client] server replied: %s\n", buf);

    close(fd);
    return 0;
}
```

Holds:

- Path/IP address of server
 - Command line input
- Port number
- Uniquely identifies a server in network

Connects to the server

Client code

```
int main(int argc, char const *argv[]) {
int sfd = socket(AF_UNIX, SOCK_STREAM, 0); // 1. create

struct sockaddr_un Saddr;

Saddr.sun_family = AF_INET;
Saddr.sun_port=htons(8000) //Server Port no is 8000

if (inet_pton(AF_INET, argv[1], &addr.sin_addr) <= 0) {
printf("\nInvalid address/ Address not supported\n");
return -1; }

if (connect(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
perror("connect"); exit(1);
}

const char *msg = "Hello from Server!";
write(fd, msg, strlen(msg)); // 3. send

char buf[300];
int n = read(fd, buf, sizeof(buf) - 1); // 4. receive reply
buf[n] = '\0';
printf("[client] server replied: %s\n", buf);

close(fd);
return 0;
```

Holds:

- Path/IP address of server
 - Command line input
- Port number
- Uniquely identifies a server in network

Connects to the server

SIGNAL

Signal is an asynchronous way to send notification to a process

- Similar to software interrupt/trap
 - Sender: notifies an event happened
 - Receiver: handles the event with its event handler
-
- **SIGINT** is a signal generated when a user presses Control-C. This will terminate the program from the terminal.
 - **SIGALRM** is generated when the timer set by the alarm function goes off.
 - **SIGABRT** is generated when a process executes the abort function.
 - **SIGSTOP** tells LINUX to pause a process to be resumed later.
 - **SIGCONT** tells LINUX to resume the processed paused earlier.
 - **SIGSEGV** is sent to a process when it has a segmentation fault.
 - **SIGKILL** is sent to a process to cause it to terminate at once.

Linux has 31 standard signal

What happens when a signal occurs?

- All signal by default comes to kernel as an exception.
- Kernel exception handler then decides whether to
 - Handle by himself or pass on to the process

On receiving a Signal, a process handles it.

There are three possibilities:

1. **Ignore it:** Process can ignore signal but not all
 - Divid by zero can not be ignored
2. **Handle it:** the process will have handler function to handle it
3. **Default action:**
 - terminate
 - terminate and dump core
 - stop or pause the program
 - resume a program paused earlier

SIGNAL Example

Ctrl-c in [Shell/Terminal](#):

- It will generate a trap to OS
- OS will stop the current running process, sending it a 2/SIGINT signal
- Signal handler for 2/SIGINT signal execute
- Default signal handler for 2/SIGINT signal exits process

Lets see some code: [signalhandling.c](#)

Next class programming:

Theory class: Memory management:ch12,13,14